

Formation Spring

Configuration XML



1

Prise en main de Spring via configuration XML

2

- ✓ Fichier XML
- ✓ Déclaration et paramétrage des beans
 - ✓ Injection par constructeur
 - ✓ Injection par setter
- ✓ Création du contexte Spring
- ✓ Récupération d'un bean
- ✓ Héritage
- ✓ List / Map
- ✓ Scope
- ✓ Properties

Paramétrage par XML

- Existe depuis les tous débuts du Spring
- Est resté le mode privilégié jusqu'à Spring 3
 - ➡ Spring 3 a introduit le paramétrage par classes annotées en `@Configuration`
- A commencé à se faire bouter en Spring 4
 - ➡ Spring 4 a introduit Spring Boot

Paramétrage par XML

➤ Avantages :

- Pas de code Java
- Tout en XML
- Peut s'explorer en plusieurs fichiers (rangés par thèmes métiers ou fonctionnelles)
- Si modifié : pas besoin de recompiler le code

➤ Inconvénients :

- Plus à la mode
- Nécessite une vraie compréhension et connaissance du Spring
- Parfois assez verbeux et lourd selon les modules du Spring
- Parfois limité dans la configuration (on n'a moins de puissance qu'avec les classes annotées en `@Configuration`)

Définition Simple - XML

➤ Définition XML de base d'un contexte Spring

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
                           http://www.springframework.org/schema/beans/spring-beans.xsd">

  <bean id="utilisateurService" class="com.france.service.UtilisateurServiceImpl"/>

</beans>
```

➤ Le code Java équivalent :

```
UtilisateurServiceImpl utilisateurService = new UtilisateurServiceImpl();
```

Définition Simple - XML

- Balise **<bean>** (enfant de **<beans>**)
 - **id** : indique l'identifiant du bean. Doit être unique au sein du context Spring. Est **important** si vous voulez faire usage de votre bean.
 - **class** : nom long de la classe à fabriquer. Est **obligatoire**.
- C'est le contexte Spring qui va se charger de l'instanciation de vos beans déclarés.
- Dans notre exemple, un constructeur **public et sans paramètre** est présent dans la classe visée.
 - Le cas échéant, Spring vous indiquera une erreur lors du chargement du contexte

Définition Simple - XML

➤ Configuration d'un bean via son constructeur

➡ Balise **<constructor-arg>** (enfant de **<bean>**)

❑ **ref** : référence à un id de bean déclaré dans le contexte Spring

```
...  
<bean id="utilisateurRepository" class="com.france.DefaultRepository"/>  
<bean id="utilisateurService" class="com.france.service.UtilisateurServiceImpl">  
  <constructor-arg ref="utilisateurRepository"/>  
</bean>  
...
```



➤ Equivalent en Java

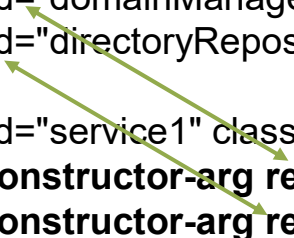
```
DefaultRepository utilisateurRepository = new DefaultRepository()  
UtilisateurServiceImpl utilisateurService = new UtilisateurServiceImpl(utilisateurRepository);
```

Définition Simple - XML

- Toujours via le constructeur, Spring permet d'injecter plusieurs dépendances pour un même bean

```
<bean id="domainManager" class="com.france.DefaultDomainManager"/>
<bean id="directoryRepository" class="com.france.DefaultRepository"/>

<bean id="service1" class="com.france.service.ServiceImpl">
  <constructor-arg ref="domainManager"/>
  <constructor-arg ref="directoryRepository"/>
</bean>
```



- Equivalent en Java

```
DefaultDomainManager domainManager = new DefaultDomainManager()
DefaultRepository directoryRepository = new DefaultRepository()
ServiceImpl service1 = new ServiceImpl(domainManager, directoryRepository);
```

- L'ordre de déclaration des **<constructor-arg>** est important !
- L'ordre de déclaration des **<bean>** n'a aucune importance !

Définition Simple - XML

- Toujours via le **constructeur**, si vous ne voulez pas passer par des références, Spring permet d'injecter des valeurs simples

- Attribut **value** de votre balise **<constructor-arg>**

```
<bean id="service2" class="com.france.service.ServiceImpl">  
  <constructor-arg value="3"/>  
  <constructor-arg value="Bonjour"/>  
</bean>
```

- Equivalent en Java

```
// En supposant qu'il existe un constructeur prenant un entier et une chaîne  
ServiceImpl service2 = new ServiceImpl(3, "Bonjour");
```

- Pour lever les ambiguïtés sur les types

- Attribut **index** de votre balise **<constructor-arg>**

```
<bean id="service1" class="com.france.service.ServiceImpl">  
  <constructor-arg index="0" value="3"/>  ← valeur scalaire indexée  
  <constructor-arg index="1" value="Bonjour"/>  
</bean>
```

Définition Simple - XML

➤ Avantages de l'injection par **constructeur** :

- L'injection par constructeur permet de définir un contrat de dépendance fort (besoin de toutes ses dépendances).
- L'injection par constructeur fait un usage concis des déclarations de beans.
- L'injection par constructeur évite le surcoût de création des méthodes « set ».

➤ Inconvénients de l'injection par **constructeur** :

- Si le bean possède de nombreuses dépendances, la liste des paramètres de construction peut être très longue.
- S'il existe plusieurs possibilités pour construire le bean, la définition des constructeurs peut devenir complexe.
- Ambiguïté des paramètres de même type.
- Problème de l'héritage et de l'appel à la méthode `super()` pour mettre à jour des données privées.

Définition Simple - XML

➤ Configuration d'un bean via injection par accesseurs

- Balise **<property>** (enfant de **<bean>**)

```
<bean id="utilisateurRepository" class="com.france.DefaultRepository"/>
<bean id="utilisateurService" class="com.france.service.UtilisateurServiceImpl">
  <property name="repository" ref="utilisateurRepository"/>
</bean>
```

- ❑ **name** : indique une méthode setXxx se trouvant dans la classe du bean
- ❑ **ref** : indique un id de bean déclaré dans le context Spring

➤ Equivalent en Java

```
DefaultRepository utilisateurRepository = new DefaultRepository();
UtilisateurService utilisateurService = new UtilisateurServiceImpl();
utilisateurService.setRepository(utilisateurRepository);
```

➤ Remarques

- Utilisation de la convention de nommage des accesseurs de la norme JavaBeans (setXxx)
- Pour les types primitifs, le conteneur Spring effectue automatiquement les conversions

Définition Simple - XML

- Tout comme pour la balise `<constructor-arg>`, dans **`<property>`**
 - **ref** : indique un id de bean déclaré dans le contexte Spring
 - **value** : indique une valeur
- **`<property>`** fait référence aux méthodes **setXxx** qui sont dans la classe. Elle ne s'occupe pas des attributs.
 - Les méthodes **setXxx** doivent être *public*
 - ❑ Sinon, Spring vous indiquera une erreur lors du chargement du contexte
- L'ordre de déclaration des `<property>` n'a aucune importance !
 - Rappel : l'ordre de déclaration des `<bean>` n'a aucune importance !

Définition Simple - XML

➤ Avantages de l'injection par **accesseurs** :

- L'injection par accesseur permet les dépendances facultatives
- L'injection par accesseur décrit les dépendances de façon claires donc plus lisible
- Les accesseurs sont hérités automatiquement
- Lève automatiquement les ambiguïtés sur les types

➤ Inconvénients de l'injection par **accesseurs**

- Notation très verbeuse
- Le contrat d'interface est trop lâche (risque de lever des exceptions de type `NullPointerException`).

Définition Simple - XML

- Quand utiliser l'injection par constructeur ?
 - Dépendances obligatoires
- Quand utiliser l'injection par les accesseurs ?
 - Dépendances facultatives
 - Le nombre de dépendances obligatoires devient très importante
- Spring permet de mélanger les deux méthodes et n'impose pas de norme.
- Utiliser le bon sens et rester cohérent.

Définition Simple - XML

- Spring permet l'injection par les deux méthodes pour un bean

```
<bean id="directoryRepository" class="com.france.fspring.DefaultRepository"/>
<bean id="domainManager" class="com.france.fspring.DefaultDomainManager"/>

<bean id="service" class="com.france.fspring.service.ServiceImpl">
  <constructor-arg ref="domainManager"/>
  <property name="directoryRepository" ref="directoryRepository"/>
</bean>
```

- Equivalent Java

```
DefaultDomainManager domainManager = new DefaultDomainManager()
DefaultRepository directoryRepository = new DefaultRepository()

ServiceImpl service = new ServiceImpl(domainManager);
service.setDirectoryRepository(directoryRepository);
```

Injection de dépendances

- Spring permet d'injecter autre chose que des beans

```
<bean id="service" class="exemple.ServiceImpl">  
  <property name="nomDeFamille" value="Durand"/>  
</bean>
```

```
public class ServiceImpl implements Service {  
  private String nomDeFamille;  
  public void setNomDeFamille(String unNomDeFamille) {  
    this.nomDeFamille = unNomDeFamille;  
  }  
  ...  
}
```

- Equivalent Java

```
ServiceImpl service = new ServiceImpl();  
service.setNomDeFamille("Durand");
```


Injection de dépendances

➤ Spring effectue le transtypage automatiquement

```
<bean id="service" class="exemple.ServiceImpl">  
  <property name="montant" value="123"/>  
</bean>
```

```
public class ServiceImpl implements Service {  
  private Integer prix;  
  public void setMontant(Integer unMontant) {  
    this.prix = unMontant;  
  }  
}
```

➤ Equivalent Java

```
ServiceImpl service = new ServiceImpl();  
service.setMontant(Integer.valueOf("123"));
```

Injection de dépendances

- Contrairement à la JSTL, Spring est **tolérant** sur les majuscules / minuscules.

```
<bean id="service" class="exemple.ServiceImpl">  
  <property name="montant" value="123"/>  
</bean>
```



```
<bean id="service" class="exemple.ServiceImpl">  
  <property name="Montant" value="123"/>  
</bean>
```



- Si Spring, ne trouve pas la méthode **setXxx** (problème de visibilité, de typage, d'id de bean introuvable ...)
 - Vous aurez une erreur lors du chargement du contexte Spring



Travaux pratiques

19

Réaliser les travaux pratiques suivants:

TP 01 : Mise en place d'un projet Spring dans Eclipse

TP 02 : Création de bean

Création d'un contexte applicatif

Présentation (1/2)

- Les contextes applicatifs Spring peuvent être initialisés dans n'importe quel environnement :
 - Test Unitaires Automatisés avec JUnit.
 - Application Web.
 - Enterprise Java Bean (EJB).
 - Application autonome (Swing, Batch).
- Le contexte est initialisé depuis un fichier disponible au sein
 - Du classpath.
 - Du système de fichier local.
 - D'une ressource externe.

Création d'un contexte applicatif

Présentation (2/2)

- L'interface la plus utilisée est
« `org.springframework.context.ApplicationContext` »

- Spring propose plusieurs implémentations :
 - `org.springframework.context.support.ClassPathXmlApplicationContext`
 - ❑ Chargement du contexte depuis une source XML présente dans le classpath
 - `org.springframework.context.support.FileSystemXmlApplicationContext`
 - ❑ Chargement du contexte depuis une source XML depuis le système de fichiers
 - `org.springframework.web.context.support.XmlWebApplicationContext`
 - ❑ Chargement du contexte depuis une source XML dans le contexte d'une application web

Création d'un contexte applicatif

Tests Unitaires Automatisés

➤ Exemple : contexte Spring avec JUnit 4 (sans les annotations Spring pour JUnit)

```
import org.junit.*;

public class TransfertServiceTests {

    private ITransfertService transfertService;

    // Code fait maison, à ne pas reproduire dans la vraie vie (voir chapitre sur Spring & JUnit)
    @Before
    public void setUp() {
        ApplicationContext context = new ClassPathXmlApplicationContext("application-context.xml");
        this.transfertService = context.getBean("transfertService", ITransfertService.class);
    }

    @Test
    public void testTransfert() {
        // Mon code de test.
    }
}
```

Création d'un contexte applicatif

Modes de chargement

- Spring offre un large éventail de possibilités pour créer un contexte applicatif
 - Via le classpath

```
new ClassPathXmlApplicationContext("applicationContext.xml");
```

- Via le système de fichiers

```
// chemin en absolu
```

```
new FileSystemXmlApplicationContext("c:/config/application-config.xml");
```

```
// chemin relatif par rapport au répertoire de travail de la JVM.
```

```
new FileSystemXmlApplicationContext("./application-config.xml");
```

Création d'un contexte applicatif

Bonnes Pratiques

- Un contexte Spring peut être initialisé à partir de plusieurs fichiers de configuration
- Permet de séparer la définition des beans en les regroupant par thématiques
 - Domaine métier
 - Accès à la base de données
 - Web services
 - etc...
- Il est conseillé d'extraire la configuration des beans métiers de ceux relatifs à l'environnement
 - Faciliter le déploiement en recette, intégration et production
 - Permet l'utilisation des Tests Unitaires Automatisés

Création d'un contexte applicatif

Bonnes Pratiques

- Par exemple, en test le serveur de base de données est MySQL et en production Oracle

- Création du contexte applicatif en environnement de test

```
ApplicationContext context = new ClassPathXmlApplicationContext(  
    "application-context.xml",  
    "mysql-database.xml");
```

- Création du contexte applicatif en environnement de production

```
ApplicationContext context = new ClassPathXmlApplicationContext(  
    "application-context.xml",  
    "oracle-database.xml");
```

- Vous pouvez séparer les chemins par une **virgule**

Création d'un contexte applicatif

Les préfixes

- Spring convertit automatiquement les chemins en ressources de type « java.net.URL »

```
ApplicationContext context = new ClassPathXmlApplicationContext(  
    "config/dao-config.xml",  
    "config/service-config.xml",  
    "file:///c:/temp/oracle-database.xml");
```

- Il existe d'autres préfixes
 - **classpath** : classpath:com/myapp/config.xml
 - **file** : file:///c:/data/config.xml
 - **http** : http://myserver/data/config.xml
- Plus globalement, dès que l'accès à une ressource quelconque est nécessaire l'usage des préfixes est possible.

Création d'un contexte applicatif

Le caractère « * » (joker)

- Spring permet de charger un ensemble de ressources par l'utilisation du caractère joker « * ».

```
new ClassPathXmlApplicationContext("classpath*:conf/*-config.xml");
```

- Tous les fichiers dont le nom se termine par « -config.xml » sont pris en compte.



- « classpath* » prend en compte l'ensemble des ressources pour au final les fusionner au sein d'un seul contexte.
 - « **classpath***:application-context.xml » - Spring charge tous les fichiers « application-context.xml » présents dans le classpath.
 - « **classpath**:application-context.xml » - Spring charge uniquement le premier fichier qu'il trouve sur le chemin.

Récupération d'instances de bean

- Vous pouvez récupérer vos beans à travers la méthode **getBean** de votre objet contexte
- Récupération d'un bean avec transtypage (« cast »)

```
ApplicationContext ctx = new ClassPathXmlApplicationContext("applicationContext.xml");  
ITransfertService service = (ITransfertService) ctx.getBean("transfertService");  
service.transfert(2956);
```

- L'instance d'un bean est récupérée du contexte par la valeur de son attribut « id » (identifiant unique).

- Depuis la version 3 de Spring, il est possible de récupérer un bean sans *cast*

```
ITransfertService service = ctx.getBean("transfertService", ITransfertService.class);  
  
// Il n'est pas nécessaire de préciser son identifiant si le bean est unique.  
ITransfertService service = ctx.getBean(ITransfertService.class);
```

Prise en main de Spring

Conclusion

- Spring instancie les beans dans l'ordre dicté par les dépendances entre beans
- Les beans peuvent être déclarés suivant un ordre quelconque
 - Spring se charge automatiquement de l'ordonnement.
- Chaque bean se voit affecter un identifiant unique par contexte
- Spring centralise la gestion du cycle de vie des instances d'objets
- Spring favorise l'utilisation des interfaces
 - Ajoute un niveau d'abstraction vis-à-vis des implémentations.
 - Gain en modularité.
- Facilite le test unitaire automatisé
 - Les implémentations sont facilement substituables.

Bonnes pratiques de configuration de beans

30

- ✓ Héritage entre beans
- ✓ Collection / Map
- ✓ Autowire XML

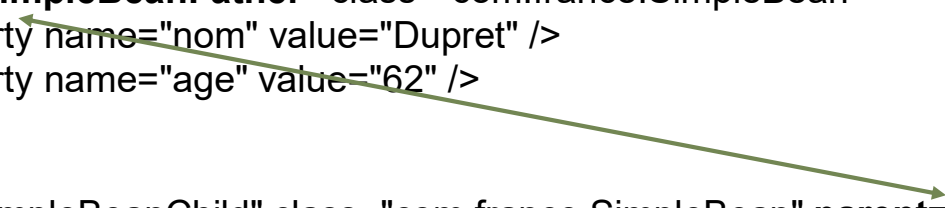
Héritage entre beans

- Il est possible de déclarer un *héritage* au niveau des balises xml, il suffit de faire usage de l'attribut **parent** de la balise **<bean>**

- Cet attribut prendra toujours un **id** de bean qui devra être présent dans le contexte Spring

```
<beans ...>
  <bean id="simpleBeanFather" class="com.france.SimpleBean">
    <property name="nom" value="Dupret" />
    <property name="age" value="62" />
  </bean>

  <bean id="simpleBeanChild" class="com.france.SimpleBean" parent="simpleBeanFather">
    <property name="age" value="35" />
  </bean>
</beans>
```



age=35
nom=Dupret



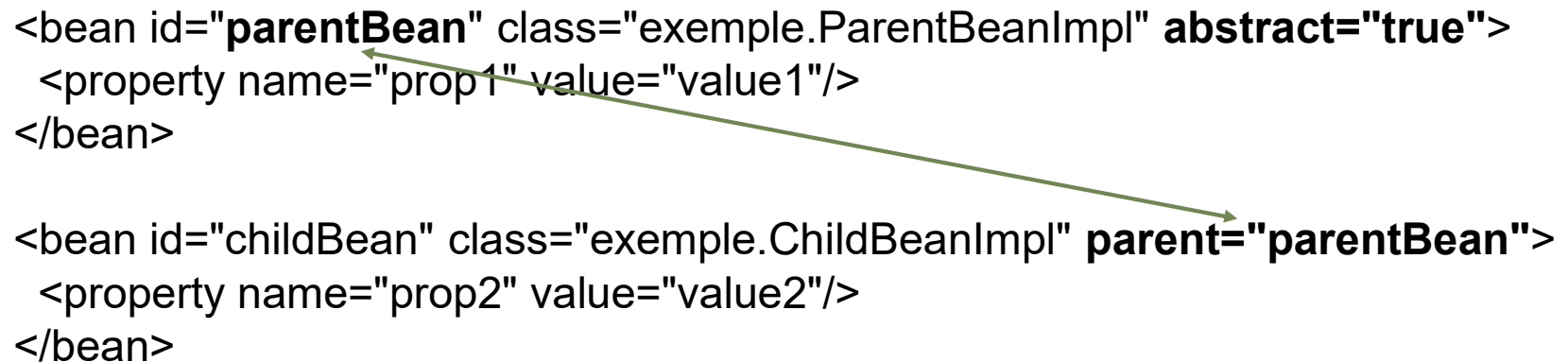
Attention! cet héritage est totalement dissocié d'un éventuel héritage Java

Définition des Beans par héritage

➤ Spring permet l'héritage entre beans

```
<bean id="parentBean" class="exemple.ParentBeanImpl" abstract="true">
  <property name="prop1" value="value1"/>
</bean>

<bean id="childBean" class="exemple.ChildBeanImpl" parent="parentBean">
  <property name="prop2" value="value2"/>
</bean>
```



- Si la valeur « **abstract** » est à « **true** » alors le bean ne peut être utilisé directement. = ne sera pas instancié par Spring.
- Bean *enfant* caractérisé par l'attribut « **parent** » pointant vers l'id du bean parent.

prop1=value1
prop2=value2

Définition des Beans par héritage

- Le bean enfant doit contenir les constructeurs et propriétés du bean parent
- Il n'y a pas obligation d'héritage au sens Java

```
public abstract class ParentBean {  
    private String firstName;  
    public ParentBean() {  
        super();  
    }  
    public void setFirstName(String a) { ... }  
}
```

← Pas d'héritage →

```
public class ChildBean {  
    private String firstName;  
    private String lastName;  
    public ChildBean() {  
        super();  
    }  
    public void setFirstName(String a) { ... }  
    public void setLastName(String b) { ... }  
}
```

Définition des Beans par héritage

- Le bean parent peut ne pas avoir de classe concrète

```
<bean id="parentBean" abstract="true">  
  <property name="firstName" value="Paul"/>  
</bean>
```

- Bean parent est utilisé comme pure template.

- Le bean enfant peut surcharger les attributs Spring du bean parent

```
<bean id="childBean" class="exemple.ChildBean" parent="parentBean">  
  <property name="lastName" value="Durand"/>  
  <property name="firstName" value="Jean"/>  
</bean>
```

- Les attributs suivants sont toujours définis par le bean enfant : depends on, autowire mode, dependency check, singleton, scope, lazy init.

- L'héritage Spring n'a pas de correspondance avec l'héritage Java

- Spring – Héritage de configuration de bean.
- Java – Héritage de classe.

Wiring des collections (1/2)

- Spring permet d'injecter des dépendances vers des collections d'objets ou des collections de valeurs
- Collections Java supportées
 - `java.util.List` `<list/>`
 - `java.util.Set` `<set/>`
 - `java.util.Map` `<map/>`
 - `java.util.Properties` `<props/>`

Wiring des collections (2/2)

➤ Wiring d'une liste

```
<property name="elements">
  <list>
    <value>item1</value>
    <value>item2</value>
    <ref bean="bean2" />
    <ref bean="bean3" />
  </list>
</property>
```

➤ Wiring d'une Map

```
<property name="elements">
  <map>
    <entry key="key1">
      <value>item1</value>
    </entry>
    <entry key="key2">
      <ref bean="bean4" />
    </entry>
  </map>
</property>
```

Injection de dépendances des collections

➤ Liste de valeurs scalaires

```
<bean id="carnetAdresse" class="exemple.CarnetAdressImpl">  
  <property name="adresses">  
    <list>  
      <value>valeur1</value>  
      <value>valeur2</value>  
      <value>valeur3</value>  
    </list>  
  </property>  
</bean>
```

➤ Equivalent Java

```
CarnetAdressImpl carnetAdresse = new CarnetAdressImpl();  
List<?> valeurs = new java.util.ArrayList<>();  
valeurs.add("valeur1");  
valeurs.add("valeur2");  
valeurs.add("valeur3");  
carnetAdresse.setAdresses(valeurs);
```

Injection de dépendances des collections

➤ Liste de beans

- On fera usage ici de l'attribut **bean** de la balise **<ref>** (enfant de **<list>**). Il indiquera l'id d'un bean chargé dans le contexte Spring

```
<bean id="carnetAdresse" class="exemple.CarnetAdresseImpl">
  <property name="adresses">
    <list>
      <ref bean="adresse1"/>
      <ref bean="adresse2"/>
      <ref bean="adresse2"/>
    </list>
  </property>
</bean>
```

➤ Les interfaces de type Set

- Identique à une liste mais avec le tag **<set>**.

Injection de dépendances des collections

➤ Les propriétés (équivalent à une Map)

➡ Utilisation des tags `<props>` et `<prop>`

```
<bean id="someObject" class="exemple.SomeObject">
  <property name="admins">
    <props>
      <prop key="key1">value1</prop>
      <prop key="key2">value2</prop>
      <prop key="key3">value3</prop>
    </props>
  </property>
</bean>
```

Injection de dépendances des collections

➤ L'interface de type « Map »

➤ Utilisation des tags <map> et <entry>

```
<bean id="someObject" class="exemple.SomeObject">
  <property name="admins">
    <map>
      <entry key="key1" value="value1"/>
      <entry key="key2" value="value2"/>
      <entry key="key3" value="value3"/>
      <entry key="key4" value-ref="idBean01"/>
      <entry key-ref="idKey02" value-ref="idBean02"/>
    </map>
  </property>
</bean>
```

➤ Vous pouvez faire usage des attribut **key-ref** ou **value-ref** afin de référencer l'id d'un bean chargé dans le contexte Spring



Travaux pratiques

41

Réaliser les travaux pratiques suivants:

TP 03 : Liste de bean

Wiring automatique

- Il est possible de laisser le conteneur Spring définir automatiquement les dépendances entre les beans

```
<bean id="myBean" class="com.MyBeanImpl" autowire="autowireType" />
```

- Spring définit 4 types d'Autowiring (valeur possible pour l'attribut *autowire*)

- **byName**

```
<bean id="myBean" class="com.MyBeanImpl" autowire="byName" />
```

- **byType**

```
<bean id="myBean" class="com.MyBeanImpl" autowire="byType" />
```

- **constructor**

```
<bean id="myBean" class="com.MyBeanImpl" autowire="constructor" />
```

- **no**

```
<bean id="myBean" class="com.MyBeanImpl" autowire="no" />
```

Exemple sans autowiring

- Il est nécessaire de préciser la propriété à injecter
 - C'est le cas "classique"

```
<beans>
  <bean id="accountService" class="com.france.services.AccountServiceImpl" />
  <bean id="customerService" class="com.france.services.CustomerServiceImpl">
    <property name="accountService" ref="accountService" />
  </bean>
</beans>
```

- Ici, il existe une méthode **setAccountService** dans la classe CustomerServiceImpl
 - ❑ Elle prend en paramètre un objet de classe AccountServiceImpl

Exemple avec autowiring...

- Spring "devine" les propriétés à injecter

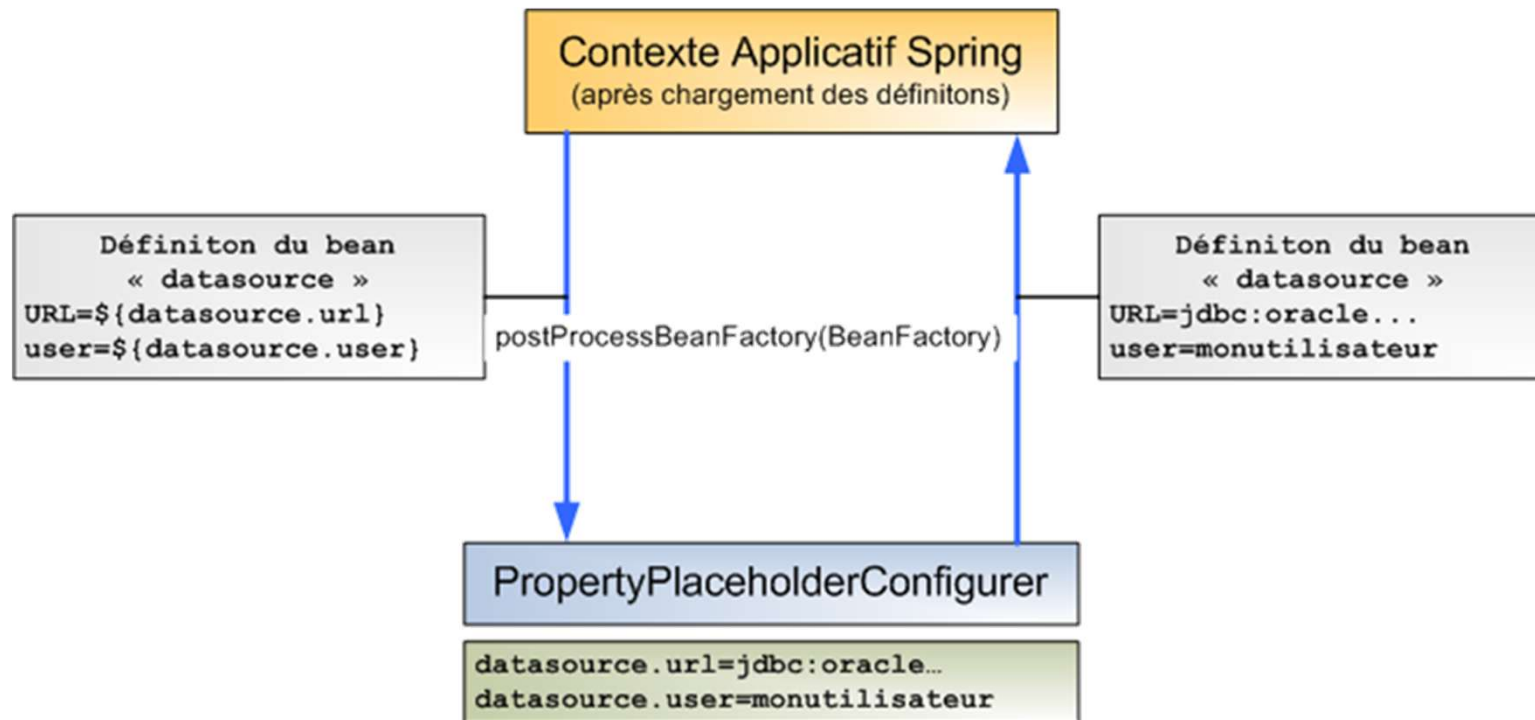
- Exemple pour **byType**

```
<beans>
  <bean id="accountService"    class="com.srv.AccountServiceImpl" />
  <bean id="customerService" class="com.srv.CustomerServiceImpl" autowire="byType" />
</beans>
```

- Ici, le Spring va regarder toutes les méthodes setXxx qui se trouvent dans votre classe CustomerServiceImpl et va tenter de chercher avec qui il pourra se lier en fonction du type de chaque méthode setXxx.
- Possibilité de mixer les déclarations automatiques et les dépendances explicites
- L'autowiring est pratique mais dangereux à l'usage : **! Attention !** au refactoring du code

Configuration via un fichier *properties*

- Externaliser des paramètres de configuration dans des fichiers propriétés
 - Utilisation du « **PropertyPlaceholderConfigurer** »



Configuration via un fichier *properties*

➤ Sans fichier properties :

```
<bean id="dataSource" class="com.oracle.jdbc.pool.OracleDataSource">  
  <property name="URL" value="jdbc:oracle.thin@localhost:1521:BANK"/>  
  <property name="user" value="monutilisateur"/>  
</bean>
```

➡ Toutes les informations sont dans le fichier XML

➤ On sort les informations dans un fichier properties

```
datasource.url=jdbc:oracle.thin@localhost:1521:BANK  
datasource.user=monutilisateur
```

➡ Structure : *clef*=valeur

Configuration via un fichier *properties*

- Dans notre fichier de configuration Spring

```
<beans>
  <!-- On charge le fichier properties -->
  <bean
class="org.springframework.beans.factory.config.PropertyPlaceholderConfigurer">
    <property name="location" value="config/configuration.properties"/>
  </bean>

  <bean id="dataSource" class="com.oracle.jdbc.pool.OracleDataSource">
    <property name="URL" value="${datasource.url}"/>
    <property name="user" value="${datasource.user}"/>
  </bean>

</beans>
```

- On place des `${clef}` dans notre fichier

Configuration via un fichier properties

- Dans un des fichiers Spring on déclare son bean **PropertyPlaceholderConfigurer**
 - Il n'a pas d'id, car on ne fera jamais un getBean dessus
 - Cet objet est **pour** le paramétrage du contexte Spring, pas pour votre application
- Le fichier properties est chargé dans le contexte Spring
 - Cette opération est faite au démarrage et avant l'instanciation de vos beans
 - Peu importe où vous placez votre bean PropertyPlaceholderConfigurer
- Vous pouvez utiliser toutes les clefs qu'il contient en utilisant la syntaxe **`${xxx}`**
 - **ATTENTION** : aux majuscules / minuscules, aux noms de vos clefs
 - ❑ Faites copier/coller de vos noms de clefs

Configuration via un fichier properties

- Depuis la version 3.1 du Spring vous pouvez aussi faire usage du **PropertySourcesPlaceholderConfigurer**
- Cet objet hérite du **PropertyPlaceholderConfigurer**, il sait gérer plus finement les propriétés d'environnement
 - <https://docs.spring.io/spring/docs/current/javadoc-api/org/springframework/core/env/ConfigurableEnvironment.html>



Travaux pratiques

50

Réaliser les travaux pratiques suivants:

TP 04 : Utilisation d'un fichier properties

Cycle de vie du contexte applicatif

Travailler avec les post-processeurs - BeanPostProcessor

- Par défaut, Spring instancie les beans lors du démarrage du Contexte Applicatif
 - L'instanciation est réalisée dans l'ordre et toutes les dépendances sont injectées.
- Dès l'injection des dépendances réalisée, la phase de post traitement peut débuter
 - Les actions de configuration plus poussées sont réalisées.
- Après la phase de post-traitement, le bean est complètement initialisé et prêt à l'emploi
 - Chaque bean est utilisé jusqu'à sa destruction.

Cycle de vie du contexte applicatif

Travailler avec les post-processeurs - BeanPostProcessor

- « BeanPostProcessor » est un point d'extension important
 - Permet de modifier une instance d'un bean après son instantiation par le conteneur.
- Spring fournit un certain nombre d'implémentations
 - Il est possible mais assez rare d'écrire soi-même une implémentation.
- Exemples d'implémentation
 - RequiredAnnotationBeanPostProcessor
 - ❑ Fourni par Spring.
 - ❑ Assure que les attributs annotés par « @Required » sont peuplés.

Cycle de vie du contexte applicatif

Travailler avec les post-processeurs - BeanPostProcessor

- Le post-traitement est réalisable via les annotations JSR-250

```
@PostConstruct  
public void init() {  
    // Code applicatif d'initialisation  
}
```

- Donne la consigne à Spring d'appeler la méthode après injection.
 - Aucune contrainte sur la visibilité ou le nom appliqué à la méthode.
- Cette méthode est conseillée
 - Pas d'adhérence au framework Spring.
 - Conforme aux spécifications JSR-250 donc portable.

Cycle de vie du contexte applicatif

Travailler avec les post-processeurs - BeanPostProcessor

- Activation de l'annotation (*ancienne méthode*)

```
<beans>
  <bean id="utilisateurService" class="exemple.UtilisateurServiceImpl"/>
  <bean class="org.springframework.context.annotation.CommonAnnotationBeanPostProcessor"/>
</beans>
```

- Automatisement détecté ; aucune autre configuration nécessaire.
- Invoque les méthodes des beans annotés avec « @PostConstruct ».

- **OU** : Alternative, faire usage de l'attribut « init-method » de la balise <bean> en XML

```
<beans>
  <bean id="utilisateurService" class="exemple.UtilisateurServiceImpl" init-method="start"/>
</beans>
```

- La valeur de l'attribut représente le nom de la méthode que vous voulez voir appeler. Elle doit être public.

- **OU** : implémenter l'interface « org.springframework.beans.factory.**InitializingBean** »

Cycle de vie du contexte applicatif

Phase de destruction (1/2)

- Le Contexte Applicatif est stoppé
 - Destruction explicite des beans selon la configuration indiquée.
 - Destruction du Contexte Applicatif ; il n'est plus utilisable.
- Un bean peut implémenter des « callback » qui sont appelés lors de la phase de destruction
 - Utilisé pour libérer des ressources
 - ❑ Pointeurs vers des fichiers
 - ❑ Sockets
- Par annotation « **@PreDestroy** »
 - Standard JSR-250

Cycle de vie du contexte applicatif

Phase de destruction (2/2)

- **OU** : faire usage de l'attribut « destroy-method » de la balise <bean> en XML.

```
<beans>  
  <bean id="utilisateurService" class="exemple.UtilisateurServiceImpl" destroy-method="stop"/>  
</beans>
```

- La valeur de l'attribut représente le nom de la méthode que vous voulez voir appeler. Elle doit être public.

- ☐ Dans cet exemple, il y a une méthode **stop()** dans la classe UtilisateurServiceImpl

- **OU** : Implémenter l'interface « org.springframework.beans.factory.**DisposableBean** ».



Travaux pratiques

57

Réaliser les travaux pratiques suivants:

TP 05 : Init et Destroy

Cycle de vie du contexte applicatif

Portées des beans Spring

- Spring définit pour chaque bean sa portée (scope)
- Les portées disponibles
 - **singleton** une seule instance par contexte. C'est la portée par **défaut**.
 - **prototype** nouvelle instance créée chaque fois que le bean est injecté.
 - **session** nouvelle instance créée par session utilisateur. Environnement Web.
 - **request** nouvelle instance créée pour chaque requête. Environnement Web.
 - **custom** définition de nouvelles règles de gestion. Programmation avancée.
- Configuration de la portée dans la définition du bean

```
<bean id="utilisateurService" class="exemple.UtilisateurServiceImpl" scope="prototype"/>
```

Beans internes

- Un bean défini à l'intérieur d'un attribut « property » ou « constructor-arg » est appelé un bean interne (inner bean)

```
<bean id="outerBean" class="exemple.OuterBeanImpl">  
  <property name="prop1">  
    <bean class="exemple.InnerBeanImpl">  
      <property name="name" value="Robert"/>  
      <property name="email" value="bob@acme.com"/>  
    </bean>  
  </property>  
</bean>
```

- Un bean interne n'a pas besoin d'identifiant (id)
 - ❑ Un bean interne ne peut pas être réutilisé par un autre bean.
- L'attribut « scope » est également ignoré.
 - ❑ Le scope d'un bean interne est toujours « prototype ».

Espaces de nom XML (namespaces) (1/3)

➤ Ancien mode de configuration XML

```
<beans>  
  <bean class="org.springframework.beans.factory.config.PropertyPlaceholderConfigurer">  
    <property name="location" value="conf/configuration.properties"/>  
  </bean>  
</beans>
```

- Seuls les attributs standards d'un bean sont disponibles
- Nécessite de connaître les détails sous-jacents de Spring

➤ Procédure actualisée de configuration XML

```
<beans>  
  <context:property-placeholder location="conf/configuration.properties"/>  
</beans>
```

- Grâce au préfixe, les attributs sont personnalisés.
- Abstrait le développeur des couches internes de Spring

Espaces de nom XML (namespaces) (2/3)

- Les espaces de noms simplifient grandement la configuration XML
 - Apportent une certaine expressivité.
 - Cachent une grande partie des détails internes de Spring.
 - Les attributs génériques d'un bean sont de trop bas niveaux.
- Spring met à disposition un certain nombre d'espaces de noms
 - aop, beans, context, j2ee, jms, tx (transaction), util.
 - Consulter l'url <http://www.springframework.org/schema/>

Espaces de nom XML (namespaces) (3/3)

- **Attention** : Pour utiliser un espace de noms, il faut changer sa déclaration XML et préciser le / les éléments nécessaires
- Exemple pour faire usage du namespace **context** :

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xmlns:context="http://www.springframework.org/schema/context"
  xsi:schemaLocation="
    http://www.springframework.org/schema/beans
    http://www.springframework.org/schema/beans/spring-beans.xsd
    http://www.springframework.org/schema/context
    http://www.springframework.org/schema/context/spring-context.xsd">

  <context:property-placeholder location="conf/configuration.properties"/>

</beans>
```



Travaux pratiques

63

Réaliser les travaux pratiques suivants:

TP 06 : Utilisation du scope et du namespace context

p namespaces - property

- « p-namespace » permet de déclarer les attributs d'un bean sans utiliser le tag « property »

```
<bean id="personService" class="com.france.fspring.service.impl.PersonServiceImpl">  
  <property name="email" value="dperucho@aol.com"/>  
</bean>
```

➤ Equivalent avec p-namespace

```
<bean id="beanId"  
  class="com.france.fspring.thenamespaces.service.impl.PersonServiceImpl"  
  p:email="bob@aol.com"/>
```

- « p-namespace » permet également de pointer vers un autre bean

```
<bean id="beanId" class="com.france.fspring.service.impl.PersonServiceImpl"  
  p:email-ref="emailBean"/>
```


p namespaces

➤ Configurer l'URI du schéma dans le tag « beans »

```
<beans xmlns="http://www.springframework.org/schema/beans"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xmlns:p="http://www.springframework.org/schema/p"
  xsi:schemaLocation="http://www.springframework.org/schema/beans
    http://www.springframework.org/schema/beans/spring-beans.xsd">

  <bean id="myDataSource" class="org.apache.commons.dbcp.BasicDataSource"
    destroy-method="close"
    p:driverClassName="com.mysql.jdbc.Driver"
    p:url="jdbc:mysql://localhost:3306/mydb"
    p:username="root"
    p:password="masterkaoli"/>

</beans>
```

➤ Il n'existe pas de XSD défini pour ce namespace.

c namespaces - constructor

- Permet de configurer l'injection par les constructeurs
 - Equivalent avec c-namespace
- Spring s'appuie sur **le nom et le type** des arguments pour réaliser l'injection.

```
public class PersonServiceImpl implements IPersonService {  
    private String email;  
    private String fname;  
    public PersonServiceImpl(String aFname, String anEmail) {  
        this.email = anEmail;  
        this.fname = aFname;  
    }  
}
```

```
<bean id="personService" class="com.service.PersonServiceImpl">  
    <constructor-arg value="Robert"/>  
    <constructor-arg value="bob@aol.com"/>  
</bean>
```

```
<bean id="personService" class="com.service.PersonServiceImpl"  
    c:aFname="toto" c:anEmail="bob@aol.com" />
```

c namespaces - constructor

- Utiliser les index s'il existe des ambiguïtés sur les arguments

```
<bean id="personService"  
      class="com.service.PersonServiceImpl"  
      c:_0="toto" c:_1="bob@aol.com"/>
```

- Le suffixe « -ref » permet d'injecter des beans
- Configurer l'URI du schéma dans le tag « beans »

```
<beans  
  xmlns="http://www.springframework.org/schema/beans"  
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"  
  xmlns:c="http://www.springframework.org/schema/c"  
  xsi:schemaLocation=" http://www.springframework.org/schema/beans  
                      http://www.springframework.org/schema/beans/spring-beans.xsd">
```

namespaces util

➤ Le tag « util » permet une écriture XML simplifiée des définitions de bean

➤ Chargement d'un fichier de propriétés

➤ Avant :

```
<bean id="jdbcConf"
      class="org.springframework.beans.factory.config.PropertiesFactoryBean">
  <property name="location" value="classpath:com/foo/jdbc-p.properties" />
</bean>
```

➤ Avec le namespace :

```
<util:properties id="jdbcConf" location="classpath:com/foo/jdbc-p.properties"/>
```

➤ Attention : ceci **n'est pas** un `PropertyPlaceholder`. Ici le fichier de propriétés aura un rôle applicatif.

namespaces util

➤ Déclaration d'une liste

```
<bean id="emails" class="org.springframework.beans.factory.config.ListFactoryBean">  
  <property name="sourceList">  
    <list>  
      <value>pechorin@hero.org</value>  
      <value>stavrogin@gov.org</value>  
      <value>porfiry@gov.org</value>  
    </list>  
  </property>  
</bean>
```

```
<util:list id="emails">  
  <value>pechorin@hero.org</value>  
  <value>stavrogin@gov.org</value>  
  <value>porfiry@gov.org</value>  
</util:list>
```

D'autres namespaces

- Spring propose d'autres espaces de noms
 - **lang** – expose les objets écrits dans un langage dynamique tel que Jruby ou Groovy.
 - **jms** – configuration des beans JMS.
 - **tx** – gestion des transactions.
 - **aop** – configuration des beans par la technologie AOP.
 - **context** – configuration du contexte Spring (ApplicationContext).
 - ...

Spring Expression Language (SpEL)

Présentation

- SpEL – Spring Expression Language
 - Langage permettant de manipuler les graphes d'objets.
 - La syntaxe se rapproche des normes OGNL et JSF EL.
 - Utilisable sur l'ensemble des briques logicielles Spring.
- SpEL peut être manipulé depuis :
 - Une application Java via « SpelExpressionParser ».
 - Les définitions de beans XML.
 - Les annotations.
- <http://docs.spring.io/spring/docs/current/spring-framework-reference/html/expressions.html>

Spring Expression Language (SpEL)

➤ Exemples XML

```
<bean id="numberGuess" class="org.springframework.samples.NumberGuess">
  <property name="randomNumber" value="#{ T(java.lang.Math).random() * 100.0 }"/>
  <!-- other properties -->
</bean>

<bean id="taxCalculator" class="org.springframework.samples.TaxCalculator">
  <property name="defaultLocale" value="#{ systemProperties['user.region'] }"/>
  <!-- other properties -->
</bean>

<bean id="numberGuess" class="org.springframework.samples.NumberGuess">
  <property name="randomNumber" value="#{ T(java.lang.Math).random() * 100.0 }"/>
  <!-- other properties -->
</bean>

<bean id="shapeGuess" class="org.springframework.samples.ShapeGuess">
  <property name="initialShapeSeed" value="#{ numberGuess.randomNumber }"/>
  <!-- other properties -->
</bean>
```


Valeurs null

➤ Valeurs nulles

- Vous pouvez faire passer des valeurs nulles via la balise **<null/>**

```
<bean id="id00" class="com.prj.sample.MaClasse">  
  <!-- fera un new MaClasse(null) -->  
  <constructor-arg><null /></constructor-arg>  
</bean>  
  
<bean id="id01" class="com.prj.sample.MaSecondeClasse">  
  <!-- fera un setNom(null) -->  
  <property name="nom"><null /></property>  
</bean>
```

Lazy instantiation

- Par défaut vos beans sont fabriqués au chargement du contexte Spring
- Vous pouvez lui demander de le faire au moment où vous en avez besoin
 - Quand Spring fera un lien avec le bean
 - Quand vous ferez un `getBean`

```
<bean id="client1" class="com.exo.Client" lazy-init="true">  
  <property name="nom" value="Dupont" />  
  <property name="prenom" value="Jean" />  
  <property name="age" value="45" />  
</bean>
```

Dépendances entre beans

- Par défaut il n'y a pas d'ordre volontaire dans la déclaration de vos beans, Spring les fabrique en fonction des liens que vous avez exprimés
- Vous pouvez lui demander de le faire en respectant un ordre
 - Le bean A doit être fabriqué **impérativement avant** le bean B

```
<bean id="A" class="com.exo.CA">  
  <property name="nom" value="Dupont" />  
</bean>  
  
<bean id="B" class="com.exo.CB" depends-on="A">  
  <property name="age" value="45" />  
</bean>
```

Import entre fichiers XML

- Plutôt que de demander aux contextes Spring de charger plusieurs fichiers, vous pouvez faire des liens entre fichiers via la balise XML **<import resource=""/>**
- Indiquez simplement dans l'attribut *resource* le nom du fichier XML à importer
- Remarque : cette technique n'est pas à privilégier



Travaux pratiques

77

Réaliser les travaux pratiques suivants:

TP 07 : Un projet complet